

Cybersecurity Certificate

Cloud Security

AWS Security Foundations, Zero Trust, Cloud-Native Threats & Defence

Prof. Dr.-Ing. Sebastian Schlesinger

HWR Berlin

Agenda

Part I — Foundations (AWS recap)

1. Cloud Security Intro
2. Shared Responsibility Model
3. IAM & Access Management (+ Modern IAM)
4. Infrastructure Security (VPC, SG, NACL)
5. Data Protection (S3, KMS, TLS)
6. Logging & Monitoring
7. Incident Response

Part II — Advanced (Master)

- A. Zero Trust in Cloud Environments
- B. Container & Kubernetes Security
- C. Supply Chain Security
- D. Threat Modeling for Cloud Architectures
- F. Cloud-Native Detection & Response

Part III — Practice

- Exercises & threat-modeling lab
- Multiple-choice self-test (12 Q)
- References

Learning goals

- Explain the **shared responsibility model** and how it shifts across IaaS / PaaS / SaaS and managed services (EC2 vs. RDS vs. Lambda vs. EKS).
- Apply **least privilege** and **defence in depth** using concrete AWS controls (IAM, SCP, VPC, SG/NACL, KMS, CloudTrail, Config).
- Reason about **Zero Trust** as the organising principle behind these controls.
- Assess **cloud-native threats**: containers/Kubernetes, software supply chain, IAM misconfiguration, data exposure.
- Perform **threat modeling** (STRIDE) on a realistic AWS architecture.
- Design **detection & response** that is event-driven and automated.

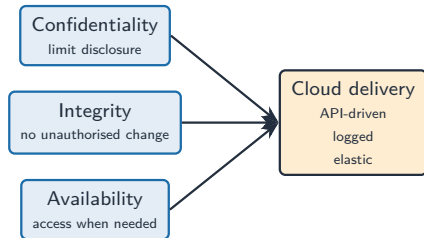
Focus on AWS, but the principles transfer to other cloud providers.

Cloud Security Intro

Section 1 of the Cloud Security module

Security is familiar — the cloud changes the *how*, not the *what*

- The objective is still **CIA**: Confidentiality, Integrity, Availability.
- What changes in the cloud:
 - Infrastructure is **API-driven** $\Rightarrow$ misconfiguration, not hardware, is the dominant risk.
 - Everything is **logged and auditable** by default-capable services.
 - Provisioning is **elastic and ephemeral** $\Rightarrow$ identity & automation replace the network perimeter.
- Cloud security properties to engineer for: **controllability, auditability, visibility, agility, automation.**



AWS security objectives mapped to services

Objective	Question it answers	Primary service
Controllability	Who can act? temporary credentials? own keys?	IAM / KMS
Auditability	Who did what, when, from where?	CloudTrail
Visibility	What exists? what changed?	Config
Agility & Automation	Reproducible, compliant deployments?	CloudFormation
Threat detection	Anomalous / malicious behaviour?	GuardDuty

Each design principle below is realised by one or more of these services.

Seven security design principles (AWS Well-Architected)

1. **Least privilege** — grant as needed, separate duties, avoid long-term credentials.
2. **Traceability** — monitor, log, audit; alert and respond automatically.
3. **Secure all layers** — defence in depth across edge, VPC, subnet, instance, app, data.
4. **Automate security** — security as code, IaC, automated guardrails.
5. **Protect data in transit & at rest** — encrypt, classify, tokenise.
6. **Keep people away from data** — reduce direct access & manual handling.
7. **Prepare for security events** — runbooks, game days, isolate & recover.

MSc framing

These principles are the *tactics*; **Zero Trust** (Part II.A) is the *strategy* that ties them together.

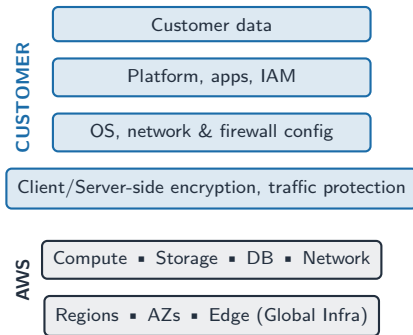
Shared Responsibility Model

Section 2 of the Cloud Security module

Shared responsibility — “of the cloud” vs. “in the cloud”

- **AWS** secures the infrastructure: hardware, global network, virtualisation, managed-service software — “security *of* the cloud”.
- **Customer** secures configuration & data: IAM, OS patching, SG/NACL rules, encryption, app logic — “security *in* the cloud”.
- The line **moves with the service model**: the more managed, the more AWS owns.

Most real-world breaches sit on the *customer* side: public S3 buckets, over-broad IAM, exposed instances.



The boundary moves with the service model

Service	Customer owns	AWS owns
EC2 (IaaS)	Guest OS, patching, SG, app, data, IAM	Hypervisor, host, network fabric
RDS (PaaS)	Schema, IAM, network, backups policy, data	DB engine patching, host OS
S3 (storage)	Bucket policy, ACL, encryption choice, data	Durability, infra, service software
Lambda (serverless)	Code, IAM role, secrets, deps	Runtime, OS, scaling, patching
EKS (managed K8s)	Worker nodes, RBAC, workloads, network policy	Control plane, etcd, API server

Rule of thumb: the customer *always* owns IAM, data classification, and “who can access what”.

Exam-style question — responsibility boundary

Question

Under the shared responsibility model, who is responsible for configuring **security group rules** that decide which ports are open on an EC2 Linux instance?

- A. AWS configures the rules.
- B. **The customer configures the rules.**
- C. Security group rules are not needed.
- D. AWS configures rules; customer opens ports.

Answer: B

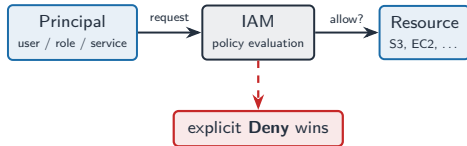
Keyword: *configuring security group rules*. SG configuration is always a customer responsibility ("in the cloud"), regardless of instance OS.

IAM & Access Management

Section 3 of the Cloud Security module

IAM: the control plane for “who can do what”

- **Authentication** — establish the *principal* (user, role, service) via credentials / federation / MFA.
- **Authorisation** — evaluate *policies* to allow or deny each request.
- A **request** carries: action, resource, principal, environment (IP, time, MFA), and resource data — all usable in **conditions**.



Building blocks

User (person/app) ■ **Group** (set of users) ■ **Role** (assumable, temporary creds via STS) ■ **Policy** (JSON document).

Policy evaluation logic — the one rule to memorise

Effective permission = intersection of *all* applicable policies:

1. Default: implicit **deny**.
2. An **Allow** in any identity- or resource-based policy can grant access...
3. ...unless a **permissions boundary** / **Service Control Policy (SCP)** fails to allow it,
4. ...and an **explicit Deny anywhere always wins**.

Common pitfall

A user with Allow * is still blocked if an SCP or boundary does not permit the action. Wildcards $\neq$ effective access.

Identity-based (attached to principal):

```
{ "Effect": "Allow",  
  "Action": ["s3:GetObject"],  
  "Resource": "arn:aws:s3:::reports/*",  
  "Condition": {  
    "Bool": {"aws:MultiFactorAuthPresent":  
              true}  
  } }
```

Resource-based (attached to resource): answers “who can touch me”, enables **cross-account** access without role assumption.

Service-Control Policy (SCP)

An organization-wide guardrail that sets the maximum permissions for AWS accounts. IAM permissions cannot exceed SCP limits.

w IAM grants permissions; SCPs define the maximum permissions that can be granted.

Guardrails: boundaries, SCPs, federation

- **Permissions boundary** — max permissions an identity *can* be granted (e.g. team cannot delegate admin roles).
- **Service Control Policy (SCP)** — org-wide ceiling across accounts (**Organizations**); cannot grant, only limit.
- **Resource policy** — access stated on the resource (S3 bucket policy, KMS key policy).
- **Federation** — trust external IdP; no local users.
- **IAM Identity Center** (ex-SSO) — central, attribute-based, short-lived sessions.
- **Cognito** — identity for web/mobile apps.
- **Directory Service** — managed AD integration.

SCP example

Deny organizations:LeaveOrganization on "*" — member accounts cannot detach.

Modern IAM: from standing access to just-in-time

- **Temporary credentials by default** — Security Token Service (STS) / roles / Identity Center sessions; kill long-lived access keys.
- **Just-in-Time (JIT) access** — elevate only for an approved window, auto-revoke.
- **Privileged Access Management (PAM)** — only allow time-boxed admin actions.
- **Break-glass accounts** — only used if usual access fails; rarely-used, heavily-monitored, MFA-hardware, alarmed on every use.
- **Workload identity** — machines get identity too: EC2 instance roles, IAM Roles for Service Accounts (IRSA) / Pod Identity (Part II.B), pods get access to AWS services without storing secrets in container.

Review & detect drift

IAM Access Analyzer — finds resources shared externally; generates least-privilege policies from CloudTrail; validates policies.

Cross-account access

Temporary role assumption rather than sharing secrets for cross-account access over sharing credentials.

Security Token Service (STS)

STS creates temporarily valid tokens containing Access Key + Secret Key + Session Token; AWS services check token validity on each request

IAM anti-patterns — and the fix

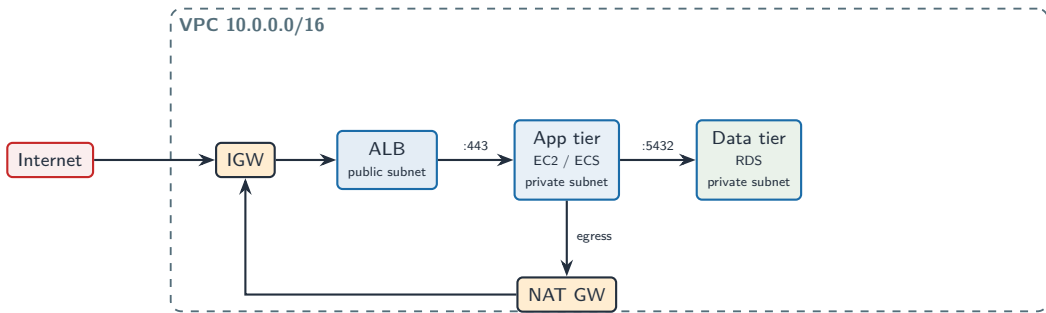
Anti-pattern ❌	Mitigation ✅
Action:"*" / Resource:"*" wildcards	Scope to actions+ARNs; generate policy from access history
Long-lived access keys in code/CI	STS / IRSA / OIDC federation; Secrets Manager; rotation
Shared / generic admin accounts	Per-identity federation; named principals in CloudTrail
Standing admin privileges	JIT elevation + PAM + session recording
iam:PassRole unscoped	Restrict which roles can be passed, to which service
No boundary on delegated roles	Permissions boundaries + SCP ceiling

Almost every cloud breach report features at least two rows of this table.

Infrastructure Security

Section 4 of the Cloud Security module

Network defence in depth: a three-tier web app



- **Public subnet:** route to Internet Gateway (IGW); only the Application Load Balancer (ALB) is exposed.
- **Private subnets:** no inbound from internet; egress via NAT only.
- Layers: IGW → Network Access Control List (NACL) → Security Group → host firewall → app.
- DB tier reachable *only* from the app-tier security group.

Security Groups vs. Network ACLs

Attribute	Security Group	Network ACL
Scope	Instance / Elastic Network Interface (ENI) level	Subnet level
Rules	Allow only	Allow <i>and</i> Deny
State	Stateful (return traffic auto)	Stateless (must allow return)
Evaluation	All rules evaluated together	Lowest rule number first
Typical use	Primary instance firewall	Coarse subnet guardrail, block bad IPs

Good SG practice

Reference *other security groups* as sources, not CIDRs — tiers authorise tiers, addresses can change.

- Also in the VPC toolbox: **route tables**, **VPC endpoints** (private S3/KMS access), **VPC Flow Logs** (Sec. 6).
- **Limit exposure**: no ('not allow') 0.0.0.0/0 on :22/:3389.

Data Protection

Section 5 of the Cloud Security module

Protecting data at rest — S3 first, because that is where it leaks

- Objects are private by default — leaks come from **misconfiguration**.
- **S3 Block Public Access** — account/bucket-level kill switch for public ACLs & policies.
- **Versioning** (+ MFA delete) — recover from deletion/ransomware.
- **Object Lock** (Write Once, Read Many (WORM), governance/compliance) — immutability for audit logs.
- Access = identity-based $\cap$ resource (bucket) policy; explicit deny wins.

Classic breach pattern

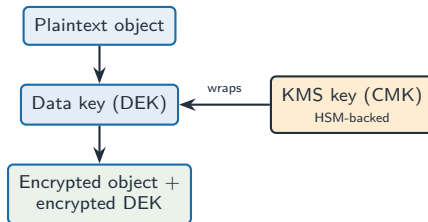
Public bucket + sensitive objects + no logging $\Rightarrow$ silent mass exfiltration. Detect with **Macie** + **Config**; threat-model in Part II.D.

Defaults to enforce

Block Public Access *on*, default encryption *on*, versioning *on*, access logging *on*.

Encryption: client-side vs. server-side, and KMS

- **Client-Side Encryption (CSE)** — you encrypt before upload; AWS never sees plaintext or keys.
- **Server-Side Encryption (SSE)** — AWS encrypts after receipt; transparent. Variants:
 - **SSE-S3** — AWS-managed keys.
 - **SSE-KMS** — KMS keys, *envelope encryption*, policy + audit.
 - **SSE-C** — you supply keys per request.
- In transit: **TLS** everywhere; **AWS Certificate Manager (ACM)** issues/rotates certs; VPC endpoints keep traffic off the public internet.



Envelope encryption: KMS protects the DEK; usage is policy-controlled & logged in CloudTrail.

Cryptographic details

$\text{DEK} \leftarrow \text{random}(256 \text{ bit})$

$\text{Ciphertext} = \text{AES}(\text{DEK}, \text{Daten})$

$\text{Wrapped_DEK} = \text{AES}(\text{KMS_Key}, \text{DEK})$

Secrets, classification & data-layer services

- **Secrets Manager** — store/rotate DB creds, API keys; no secrets in code/config.
- **KMS** — centralised key lifecycle & usage policy.
- **ACM** (+ Private CA) — TLS certificate lifecycle.
- **Macie** — discover & classify PII in S3; flag public/unencrypted/shared buckets.
- **Presigned URLs** — time-boxed, scoped object access without IAM users.
- **Least privilege** on data: enforce encryption on every PUT.

Presigned URL

A URL with embedded temporary credentials that grants access to a specific S3 object for a limited time, without needing an IAM user or role.

Logging & Monitoring

Section 6 of the Cloud Security module

Capture everything: the cloud audit substrate

- **CloudTrail** — *who did what, when, from where* (API audit). The single most important security log.
- **CloudWatch** — metrics, logs, alarms, dashboards (operational health).
- **Config** — resource inventory + configuration history + rule evaluation (“what exists / what changed”).
- **EventBridge** — event bus that routes signals to automated responders.

Built-in service logs

VPC Flow Logs (network), **S3 server access logs**, **ELB access logs**, **DNS / Route 53 logs**.

CloudTrail vs. CloudWatch

CloudTrail = *accountability* (who/what); CloudWatch = *behaviour* (anomaly/alarm). Combine: CloudTrail event → EventBridge rule → alarm/automation.

EventBridge vs. SQS

SQS: like mailbox, pull-based; EventBridge: like pub/sub, push-based. EventBridge for real-time, SQS for decoupling.

Posture management & aggregation

- **Security Hub** — aggregates findings (GuardDuty, Inspector, Macie, partners) into one normalised view + best-practice standards scoring.
- **Config** rules — continuous compliance (e.g. “no public S3”, “encryption on”, “no 0.0.0.0/0 SSH”) with auto-remediation hooks.
- **Trusted Advisor** — account-wide best-practice checks.

Reading a CloudTrail event (key fields)

`userIdentity` (who) ■ `eventName` (what, e.g. `StopInstances`) ■ `eventTime`, `sourceIPAddress`, `userAgent` (when/where) ■ `requestParameters` / `responseElements` (detail).

Equivalences

SIEM $\approx$ CloudTrail + CloudWatch + Logs

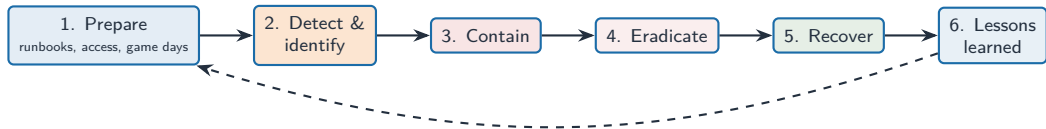
SOAR $\approx$ EventBridge + Lambda/Step Functions

Security Hub + Config rules $\approx$ vulnerability management / posture management platform

Incident Response

Section 7 of the Cloud Security module

IR in the cloud: phases & supporting services



Discovery / recognition

- **GuardDuty** (threat detection), **Inspector** (vulns), **Config** (drift), **CloudWatch**, **Shield** (DDoS).

Resolution / recovery

- **Systems Manager** (act on fleet), **CloudFormation** (rebuild forensic VPC), **Lambda/Step Functions/SNS** (event-driven automation).

IR best practices — and why automation matters

- Identify key people, external resources, tooling *before* the incident.
- **Pre-provision** break-glass access & forensic tooling.
- **Automate containment** — isolate an instance/role in seconds, not hours.
- Write **runbooks**; rehearse with **game days** in a safe environment.
- Capture evidence *before* you terminate (snapshots, memory, logs).

Cloud advantage

Compromised host? Don't clean it — **isolate, snapshot, redeploy from IaC**. Immutable infrastructure makes eradication a re-deploy.

Forward link

Part II.F turns these phases into four concrete, event-driven **playbooks**.

Part II

Advanced Cloud Security

Zero Trust ▪ Containers/Kubernetes ▪ Supply Chain ▪ Threat Modeling ▪ Detection & Response

Each topic is connected back to the AWS controls from Part I.

A. Zero Trust in Cloud Environments

Section 8 of the Cloud Security module

Zero Trust: never trust, always verify

- **Core principle:** no implicit trust from network location — every request is authenticated, authorised & encrypted.
- **Identity is the new perimeter** — controls follow the principal (user *and* workload), not the IP subnet.
- **Continuous verification** — trust is re-evaluated per request using identity, device/workload posture, and context (time, geo, risk).
- **Assume breach** — minimise blast radius via least privilege & segmentation.

Reference model: NIST SP 800-207.



Trust re-evaluated on every request.

Perimeter-based vs. Zero Trust

	Classic perimeter	Zero Trust
Trust anchor	Network location (“inside = trusted”)	Verified identity + context
Default stance	Allow within perimeter	Deny; allow per verified request
Lateral movement	Easy once inside	Contained by segmentation
Enforcement	Firewall at the edge	Distributed, per-resource
Credentials	Often long-lived	Short-lived, continuously re-checked

Why the cloud forces this

There is no single edge to defend: APIs are internet-reachable, workloads are ephemeral, identities are many. The “castle wall” dissolves.

Zero Trust pillars realised with existing AWS controls

Zero Trust pillar	AWS realisation (from Part I)
Strong identity, least privilege	IAM roles, permissions boundaries, SCPs; STS short-lived creds
Continuous verification / audit	CloudTrail (every API call) + Config (state) + conditions
Micro-segmentation	Security Groups (SG-references-SG), private subnets, VPC endpoints
Workload identity	EC2 instance roles, EKS IRSA / Pod Identity (Sec. B)
Encryption everywhere	TLS in transit, KMS at rest — no plaintext on the wire
Adaptive / risk signals	GuardDuty findings feeding policy / response (Sec. F)

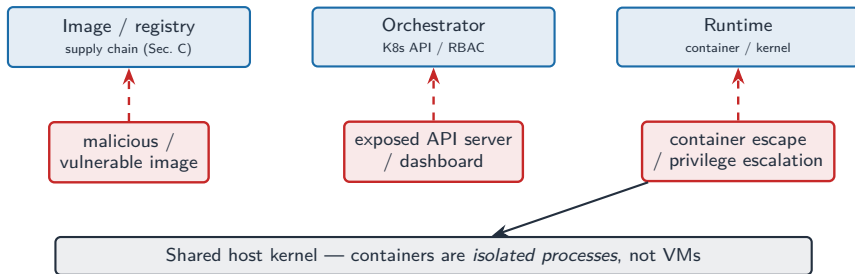
 [Link to existing AWS concepts](#)

Zero Trust is not a product to buy — it is how you compose IAM, VPC, SG, CloudTrail and Config. The

B. Container & Kubernetes Security

Section 9 of the Cloud Security module

Container threat model: where the attack surface lives



- Containers share the host kernel $\Rightarrow$ a kernel/runtime escape breaks isolation.
- Four control points: **image**, **registry**, **orchestrator**, **runtime** — secure all four (defence in depth).

Container Escape

Container escape, e.g. via kernel vulnerabilities (patching!), over-privileged containers, host file system is mounted, Linux too many capabilities, runtime weaknesses. Mitigate with least privilege, seccomp/AppArmor, no privileged containers, runtime detection (Falco).

Kubernetes security building blocks

- **RBAC** — bind subjects (users, groups, service accounts) to verbs on resources; least privilege at the API.
- **Service Accounts** — workload identity inside the cluster; do *not* auto-mount tokens you don't need.
- **Admission control** — validate/mutate objects at create time (e.g. OPA/Gatekeeper, Kyverno, Pod Security Admission).
- **Network Policies** — default-deny pod-to-pod; the SG/NACL analogue inside the cluster.
- **Secrets** — avoid plaintext in manifests; integrate KMS / external secret stores; encrypt etcd at rest.
- **Runtime security** — detect anomalous syscalls/process behaviour (e.g. Falco), enforce seccomp/AppArmor.
- **Image security** — scan, pin digests, use minimal/distroless bases (Sec. C).

Mental model

RBAC $\approx$ IAM, NetworkPolicy $\approx$ Security Group, Admission $\approx$ Config rule / SCP guardrail.

EKS specifics: who owns what, and workload identity

- **Control-plane responsibility:** AWS runs/patches the API server & etcd; *you* own worker nodes, RBAC, workloads, network policy.
- **IRSA** (IAM Roles for Service Accounts) — map a K8s service account to an IAM role via OIDC; pods get **scoped, short-lived** AWS credentials.
- **EKS Pod Identity** — newer, simpler association of IAM roles to pods.
- **Result:** *no static AWS keys in containers* — workload identity, Zero Trust style.

RBAC: read-only pods, one namespace

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata: { namespace: payments }
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]      # no
                                     create/delete
```

Link to existing AWS concepts

EKS extends the shared responsibility model: control plane = AWS, data plane = you. IRSA reuses IAM + STS from Part I.

Typical attacks → best-practice mitigations

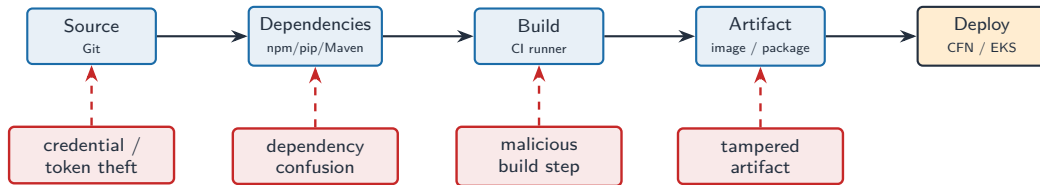
Attack ❌	Mitigation ✅
Malicious / vulnerable image	Scan in CI & registry; pin digests; signed images (Sec. C)
Exposed dashboard / API server	No public API endpoint; authn+RBAC; network policy; admission
Privilege escalation in cluster	Least-privilege RBAC; no <code>cluster-admin</code> bindings; PSA
Container escape	Non-root + read-only rootfs; drop caps; seccomp/AppArmor; no privileged pods
Credential theft from pod	IRSA/Pod Identity (no static keys); short-lived tokens
Resource exhaustion / crypto-mining	Resource limits/quotas; runtime detection (Falco)

Baseline hardening: minimal images, non-root, read-only filesystem, dropped capabilities, resource limits, image scanning.

C. Supply Chain Security

Section 10 of the Cloud Security module

CI/CD is production access — and a prime attack surface



- A pipeline with deploy rights is a privileged identity — treat its credentials accordingly.
- Trust must be **verifiable end-to-end**: source → build → artifact → deploy.

Threats & defences across the chain

Threats

- **Dependency confusion** — internal package name resolves to attacker's public package.
- **Compromised packages** — typosquatting, hijacked maintainer, malicious update.
- **Malicious build steps** — poisoned CI config, compromised runner/plugin.
- **Secrets in pipelines** — tokens in logs, env vars, or committed files.
- **Tampered artifacts** — swapped image between build and deploy.

Defences

- **Software Bill of Materials (SBOM)** — inventory every component (CycloneDX/SPDX); track with Dependency-Track.
- **Signing & verification** — sign commits, artifacts, images (Sigstore/cosign); verify at admission.
- **Provenance & Supply-chain Levels for Software Artifacts (SLSA)** — attest *how/where* an artifact was built; raise SLSA level.
- **Reproducible builds** — same input $\Rightarrow$ same output (detect tampering).
- **Policy-as-Code** — gate deploys on scan/sign/provenance checks (OPA/Kyverno).
- **Secrets hygiene** — OIDC to cloud, no long-lived keys; scoped, ephemeral tokens.

Anchoring supply-chain controls in AWS

- **Image signing/verification** — sign in CI, store in **ECR**, verify with an **admission controller** on EKS before a pod runs.
- **IaC integrity** — **CloudFormation**/Terraform reviewed, scanned (cfn-guard, checkov), and policy-gated; drift caught by **Config**.
- **Pipeline identity** — **GitHub Actions**/**GitLab CI** authenticate to AWS via **OIDC** → short-lived role, never stored access keys.
- **Secret detection** — scan repos/pipelines; route findings to the “exposed secret” playbook (Sec. F).
- **Vulnerability scanning** — **Inspector** scans ECR images continuously.

Link to existing AWS concepts

The pipeline becomes a Zero-Trust principal: it proves its identity (OIDC), gets least-privilege short-lived credentials, and every artifact it produces is signed & attested.

D. Threat Modeling for Cloud Architectures

Section 11 of the Cloud Security module

Threat modeling: structure the question “what can go wrong?”

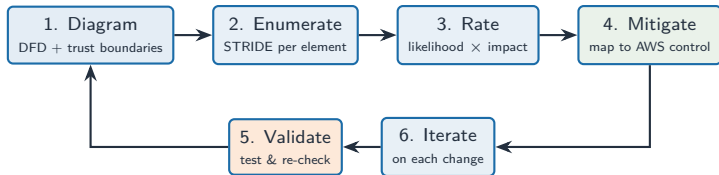
Four guiding questions (Shostack):

1. What are we building? (**assets, data flows**)
2. What can go wrong? (**threats**)
3. What will we do about it? (**mitigations**)
4. Did we do a good job? (**validate**)

Identify: **assets, trust boundaries, entry points, data flows.**

STRIDE for cloud	
Threat	Cloud example
Spoofing	Stolen IAM keys, assumed role
Tampering	Modified S3 object, poisoned image
Repudiation	CloudTrail disabled, no logs
Info disclosure	Public bucket, unencrypted traffic
Denial of service	Resource exhaustion, no quotas
Elevation of priv	Over-broad IAM, PassRole

A repeatable threat-modeling workflow



- Do it **early** (design) and **continuously** (every architecture change / new entry point).
- Output is a living artifact: threats → owners → controls → verification status.
- Pair with **Config** rules so mitigations are **enforced**, not just documented.

Exercise — “Identify threats and mitigations”

Scenario

A startup deploys: a public **ALB** → **EC2** app (role allows `s3:*`) → **RDS**. Static files live in an **S3** bucket with ACL “public-read”. SSH (:22) is open to 0.0.0.0/0. CloudTrail is *not* enabled. Secrets sit in an EC2 user-data script.

Task (15 min, in pairs)

1. Draw the DFD and mark trust boundaries.
2. Find ≥ 5 threats; classify with STRIDE.
3. Propose one AWS mitigation per threat.
4. Rank by likelihood $\times$ impact.

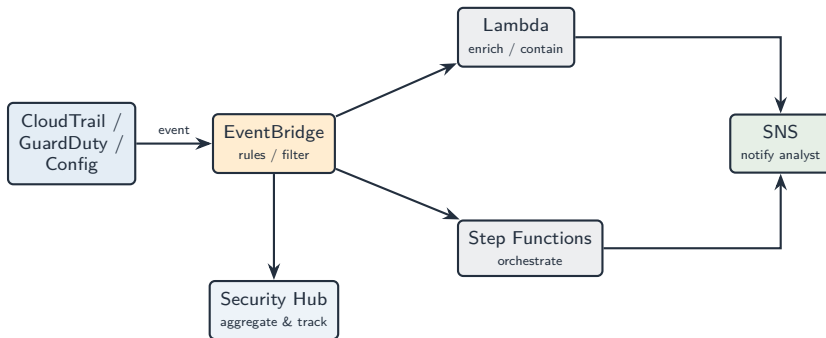
Seed answers (don't reveal first)

- Public bucket (I) → Block Public Access + bucket policy.
- `s3:*` role (E) → scope to one bucket/prefix.
- Open SSH (S/E) → SG to bastion/SSM only; no :22 to world.
- No CloudTrail (R) → enable org-wide trail.
- Secret in user-data (I) → Secrets Manager + IAM role.

F. Cloud-Native Detection & Response

Section 12 of the Cloud Security module

Detection engineering, event-driven response



- **Signal** (CloudTrail/GuardDuty/Config) → **route** (EventBridge) → **act** (Lambda/Step Functions) → **notify** (SNS) → **track** (Security Hub).
- Detection-as-code: rules are version-controlled, tested, and tuned — not click-ops.

Detection services & fighting alert fatigue

- **GuardDuty** — ML/threat-intel detection (recon, crypto-mining, creds exfil).
- **Security Hub** — normalise & aggregate findings + posture scoring.
- **Config** — config drift & compliance rules.
- **Inspector** — vulnerability findings (EC2, ECR images, Lambda).
- **Macie** — sensitive-data exposure in S3.

Alert fatigue is a security risk

Too many low-value alerts $\Rightarrow$ real ones are missed.

Prioritise by:

- **Severity** $\times$ **asset criticality** $\times$ **exploitability**.
- **Deduplicate** & correlate (Security Hub).
- **Auto-handle** the known-benign; auto-contain the unambiguous.
- Track **MTTD** / **MTTR**; tune noisy rules.

Playbooks (1–2): public S3 bucket & suspicious IAM activity

1. Public S3 bucket detected

Trigger: Config rule / Macie / GuardDuty

- **Contain:** Lambda re-applies Block Public Access; remove public statement.
- **Eradicate:** find root cause (IaC? manual?); fix template; scan for siblings.
- **Recover:** verify access logs for exfiltration window; rotate exposed data/keys.
- **Lessons:** add Config guardrail + SCP; alert on PutBucketAcl.

2. Suspicious IAM activity

Trigger: GuardDuty (anomalous API / new region / creds use)

- **Contain:** attach deny-all boundary / disable keys; revoke active sessions.
- **Eradicate:** identify entry (leaked key? phishing?); remove backdoor users/policies.
- **Recover:** rotate credentials; restore least-privilege; review CloudTrail timeline.
- **Lessons:** enforce MFA, kill long-lived keys, add Access Analyzer review.

Each step is automatable: EventBridge rule → Lambda/Step Functions → SNS to the on-call analyst.

Playbooks (3–4): compromised EC2 & exposed secret

3. Compromised EC2 instance

Trigger: GuardDuty (C2 / mining / port scan)

- **Contain:** swap to isolation SG (no egress); deny instance-role; snapshot for forensics.
- **Eradicate:** terminate; rebuild from clean AMI via CloudFormation (immutable).
- **Recover:** redeploy in clean subnet; verify; restore from backup if needed.
- **Lessons:** patch baseline image; add Inspector scan; review SG exposure.

4. Exposed secret in a repository

Trigger: secret scanner / push protection

- **Contain:** revoke/rotate the secret *immediately* (assume compromised).
- **Eradicate:** purge from history; remove from artifacts/logs; find reuse.
- **Recover:** issue new secret via Secrets Manager; audit CloudTrail for misuse.
- **Lessons:** pre-commit + CI secret scanning; OIDC instead of static keys (Sec. C/E).

Link to existing AWS concepts

Every playbook reuses Part-I services (CloudTrail, Config, GuardDuty, KMS, Secrets Manager, CloudFormation) — response is *composition*, not new tooling.



Part III

Exercises, Self-Test & References

Exercises & Self-Test

Section 13 of the Cloud Security module

Self-test — multiple choice (1–4)

- Q1.** Under the shared responsibility model for **EKS**, who is responsible for the Kubernetes *API server* and *etcd*?
A) Customer B) AWS C) Shared equally D) The CNI plugin vendor
- Q2.** The single most important principle of Zero Trust is:
A) Encrypt all disks B) Never trust, always verify C) Use a bigger firewall D) Put everything in a private subnet
- Q3.** In IAM policy evaluation, which statement is always decisive?
A) An explicit Allow B) The most recent policy C) An explicit Deny D) The resource-based policy
- Q4.** **Dependency confusion** is an attack in which:
A) Two libraries share a function name B) An internal package name is resolved to a malicious public package
C) A container cannot find its base image D) Two pods get the same service account

Self-test — multiple choice (5–8)

Q5. Which control best prevents *static AWS credentials* inside EKS pods?

- A) Network policies
- B) IRSA / Pod Identity
- C) A bigger security group
- D) Read-only root filesystem

Q6. A **permissions boundary** is used to:

- A) Grant new permissions
- B) Cap the maximum permissions an identity can have
- C) Encrypt IAM policies
- D) Replace SCPs

Q7. Which AWS service answers “*who did what, when, from where*”?

- A) CloudWatch
- B) Config
- C) CloudTrail
- D) Trusted Advisor

Q8. **SLSA** primarily provides:

- A) A container runtime
- B) Build provenance / supply-chain integrity levels
- C) A network firewall
- D) A secrets vault

Self-test — multiple choice (9–12)

Q9. In STRIDE, a *public S3 bucket* exposing PII is primarily an instance of:

- A) Spoofing
- B) Tampering
- C) Information disclosure
- D) Repudiation

Q10. The first containment action for an *exposed secret in a repo* is:

- A) Open an incident ticket
- B) Rotate/revoke the secret immediately
- C) Delete the repository
- D) Email all developers

Q11. Which Kubernetes mechanism is the closest analogue to an AWS *security group*?

- A) RBAC Role
- B) NetworkPolicy
- C) Admission controller
- D) Service account

Q12. To reduce **alert fatigue**, the best first step is:

- A) Disable GuardDuty
- B) Forward all alerts to email
- C) Prioritise by severity × asset criticality & deduplicate
- D) Increase log retention

Self-test — answer key

Q	Answer
Q1	B — AWS owns the control plane
Q2	B — never trust, always verify
Q3	C — explicit Deny always wins
Q4	B — internal name → public pkg
Q5	B — IRSA / Pod Identity
Q6	B — caps maximum permissions

Q	Answer
Q7	C — CloudTrail
Q8	B — build provenance / integrity
Q9	C — information disclosure
Q10	B — rotate/revoke immediately
Q11	B — NetworkPolicy
Q12	C — prioritise & deduplicate

Score 10+/12 ⇒ ready for the lab; review the linked section otherwise.

References

Section 14 of the Cloud Security module

Key takeaways

- The customer owns **identity, configuration and data** — that is where breaches happen.
- **Zero Trust** is the strategy; IAM, VPC, SG, CloudTrail, Config are the tactics.
- **Containers/Kubernetes** add image, orchestrator and runtime attack surfaces; EKS extends the shared-responsibility line at the control plane.
- The **software supply chain** is production access: sign, attest (SLSA), gate with policy-as-code.
- **Threat modeling** (STRIDE) turns architecture into a prioritised control list.
- **Detection & response** is event-driven and automated: signal → route → act → notify.

Same AWS services throughout — composed differently for prevention, detection and response.

References & further reading

- AWS Well-Architected Framework — *Security Pillar*. AWS, current edition.
- NIST SP 800-207 — *Zero Trust Architecture*. NIST, 2020.
- CIS *Kubernetes Benchmark* & Kubernetes docs — *Securing a Cluster / Security Best Practices*.
- OWASP *Kubernetes Top 10*. OWASP Foundation.
- SLSA — *Supply-chain Levels for Software Artifacts Framework* (slsa.dev).
- OWASP *Software Component Verification Standard (SCVS)*; CycloneDX / SPDX SBOM; OWASP *Dependency-Track*.
- MITRE ATT&CK® — *Cloud Matrix* (IaaS, Containers).
- Cloud Security Alliance — *Cloud Controls Matrix (CCM)*.
- Sigstore / cosign — artifact & container signing; in-toto provenance.
- Shostack, A. — *Threat Modeling: Designing for Security*. Wiley, 2014.
- AWS Academy — *Cloud Security Foundations* (base deck for this module).

Thank you

Questions, threat models, and counter-examples welcome.

Prof. Dr.-Ing. Sebastian Schlesinger ■ HWR Berlin ■ MSc Cybersecurity